

Relatório Técnico — Trabalho 4: Comunicação Indireta (Publish-Subscribe)

1. Contextualização do Projeto

O projeto da disciplina consiste em um sistema distribuído para gestão de estoque de uma clínica veterinária, desenvolvido progressivamente ao longo dos trabalhos. No Trabalho 1, a comunicação foi implementada via sockets TCP com protocolo binário proprietário. No Trabalho 2 (RMI), adotou-se a invocação remota de métodos com serialização JSON. O Trabalho 3 substituiu a comunicação de baixo nível por uma **API REST** construída com o framework Javalin (Java), exposta na porta 8080, com clientes implementados em C++ (libcurl + nlohmann/json) e Python (requests).

O Trabalho 4 evolui essa arquitetura ao introduzir um nível de **comunicação indireta** na forma de um sistema Publish-Subscribe, eliminando o acoplamento direto ponto a ponto que caracterizava os trabalhos anteriores.

2. Justificativa da Escolha: Publish-Subscribe

A abordagem Publish-Subscribe (Opção B) foi selecionada por ser a mais adequada ao domínio do sistema. Em uma clínica veterinária, eventos como vencimentos de vacinas, recalls de lotes e necessidades de reposição de estoque são naturalmente **assíncronos e multi-destinatários**: um mesmo evento de vencimento deve notificar simultaneamente o médico veterinário, o estoquista e o técnico de laboratório, sem que o sistema que detectou o vencimento precise conhecer cada um desses atores.

Além disso, o domínio exige **desacoplamento temporal**: um médico pode estar atendendo um paciente (offline) no momento em que um recall é emitido; o evento deve ser retido e entregue quando ele retornar ao sistema. Essa propriedade é inviável com chamadas REST diretas.

As demais opções foram descartadas pelas seguintes razões: **Multicast** exigiria que todos os receptores estivessem online simultaneamente; **Filas de Mensagens** são orientadas a um único consumidor por mensagem, inadequadas para notificações broadcast; **Espaço de Tuplas** adiciona complexidade desnecessária sem benefício claro para o cenário de notificações.

3. Arquitetura do Sistema

3.1 Estrutura de Componentes

O sistema é composto por quatro pacotes Java e um conjunto de tópicos que definem o contrato de comunicação:

- `broker.Broker` — intermediário central (singleton thread-safe). Gerencia assinaturas, roteia eventos e mantém filas de retenção por (tópico, assinante).
- `evento.Evento` — mensagem publicada, contendo tópico, conteúdo, publicador e timestamp de criação.

- `publicador.PublicadorClinica` — componente que emite eventos no broker. Não conhece nenhum assinante.
- `assinante.AssinanteVeterinario` — componente que se registra em tópicos e recebe callbacks assíncronos. Suporta operações `ficarOffline()` e `voltarOnline()`.

3.2 Tópicos Disponíveis

O sistema define cinco tópicos, implementados como enum Java (`Evento.Topico`):

Tópico	Significado	Assinantes
VENCIMENTO	Produto próximo ou já vencido	Dr. Joao, Estoquista
RECALL	Lote recolhido pelo fabricante	Dr. Joao
REPOSICAO	Produto abaixo do estoque mínimo	Tecnico, Estoquista
CADASTRO	Novo produto cadastrado no sistema	Tecnico, Estoquista
REMOCAO	Produto removido do estoque	Tecnico

3.3 Mecanismo de Retenção Offline

O Broker mantém uma estrutura `Map<String, Queue<Evento>> filaRetencao`, indexada pela chave composta `"topico:nomeAssinante"`. Quando um evento é publicado e o assinante está marcado como offline no conjunto `Set<String> offline`, o evento é enfileirado em vez de entregue. Ao retornar online (`marcarOnline()`), o broker percorre todos os tópicos, recupera as filas pendentes e as entrega assincronamente via `ExecutorService`.

O mesmo mecanismo é acionado na reassinatura: ao chamar `assinar()`, eventos retidos anteriores ao registro são entregues imediatamente, implementando assim o **desacoplamento temporal** completo exigido pelo enunciado.

4. Demonstração dos Desacoplamentos

4.1 Desacoplamento Espacial

O `PublicadorClinica` invoca exclusivamente `broker.publicar(evento)`, sem nenhuma referência a assinantes. O código do publicador não importa, não instancia e não conhece as classes `AssinanteVeterinario`. Novos assinantes podem ser adicionados ou removidos em tempo de execução sem qualquer alteração no publicador.

4.2 Desacoplamento Temporal

O `DemoT4.java` demonstra o cenário: o assinante "Tecnico" é marcado offline via `ficarOffline()`; enquanto offline, três eventos são publicados (CADASTRO, REMOCAO, REPOSICAO). O publicador continua operando normalmente, sem erros. Ao retornar online via `voltarOnline()`, os três eventos retidos são entregues automaticamente pelo broker.

5. Análise: Flexibilidade vs. Overhead

5.1 Ganhos de Flexibilidade

- Adição de novos assinantes sem modificar publicadores ou outros assinantes.
- Tolerância à indisponibilidade temporária de componentes receptores.
- Filtro por tópico: cada componente recebe apenas os eventos de seu interesse.
- Múltiplos assinantes recebem o mesmo evento em uma única publicação.

5.2 Overhead Introduzido

A introdução do Broker como intermediário gera três categorias de overhead:

- Latência adicional: cada evento passa pelo Broker antes de chegar ao assinante. Na implementação atual (in-process, Java), esse overhead é desprezível. Em uma implementação distribuída (Broker em processo separado), seria da ordem da latência de rede.
- Consumo de memória: eventos retidos para assinantes offline ficam acumulados nas filas do Broker. Em sistemas de alta frequência, isso pode pressionar o heap da JVM.
- Complexidade de gerenciamento: o Broker precisa detectar assinantes que nunca retornam (evitar memory leak). A solução convencional é TTL (time-to-live) nos eventos retidos, não implementado nesta versão.

5.3 Estratégias de Mitigação

- Entrega assíncrona via ExecutorService (já implementada): o broker não bloqueia na entrega, evitando que um assinante lento atrase os demais.
- TTL nos eventos retidos: descartar eventos com mais de N horas de retenção, limitando o crescimento das filas.
- Limite de fila por assinante: definir um tamanho máximo; ao atingir, descartar os mais antigos (política LRU).
- Broker externo em produção: usar Apache Kafka ou RabbitMQ, que oferecem persistência em disco, replicação e entrega garantida.

6. Comparativo com a Arquitetura do T3

Propriedade	T3 — API REST (comunicação direta)	T4 — Pub-Sub (comunicação indireta)
Acoplamento espacial	Cliente conhece IP e porta do servidor	Publicador não conhece nenhum assinante
Acoplamento temporal	Ambos precisam estar online simultaneamente	Eventos retidos para assinantes offline
Falha do receptor	Publicador recebe erro imediatamente	Evento fica retido no broker até retorno
Adição de novos consumidores	Requer alteração no publicador	Basta assinar o tópico — publicador não muda

A tabela evidencia que a comunicação indireta via Publish-Subscribe resolve os dois problemas centrais de acoplamento da arquitetura REST: o publicador deixa de precisar do

endereço do receptor (acoplamento espacial) e pode operar mesmo quando os receptores estão temporariamente indisponíveis (acoplamento temporal).

7. Referência: Endpoints da API REST (T3)

O servidor Java (Javalin, porta 8080) expõe os seguintes endpoints, organizados em três objetos distribuídos — ProdutoRecurso, EstoqueRecurso e RelatorioRecurso:

Método	Endpoint	Recurso	Descrição
GET	/api/produtos	ProdutoRecurso	Listar todos os produtos
GET	/api/produtos/{id}	ProdutoRecurso	Buscar produto por ID
GET	/api/produtos/especie/{e}	ProdutoRecurso	Buscar por espécie animal
GET	/api/produtos/vencidos	ProdutoRecurso	Listar produtos vencidos
GET	/api/produtos/valor-total	ProdutoRecurso	Calcular valor total do estoque
POST	/api/produtos	ProdutoRecurso	Cadastrar novo produto
DELETE	/api/produtos/{id}	ProdutoRecurso	Remover produto
GET	/api/estoques	EstoqueRecurso	Listar todos os estoques
GET	/api/estoques/vencidos	EstoqueRecurso	Alertar produtos vencidos
POST	/api/estoques/{id}/entrada/{p}	EstoqueRecurso	Entrada de produto no estoque
DELETE	/api/estoques/{id}/saida/{p}	EstoqueRecurso	Saída de produto do estoque
GET	/api/relatorios/geral	RelatorioRecurso	Relatório consolidado geral
GET	/api/relatorios/saude-estoque	RelatorioRecurso	Saúde do estoque
GET	/api/relatorios/especie/{e}	RelatorioRecurso	Relatório por espécie animal

Os clientes implementados foram: **C++** (libcurl + nlohmann/json, operações GET/POST/DELETE) e **Python** (biblioteca requests, cobrindo todos os 15 endpoints).

8. Conclusão

O Trabalho 4 demonstrou na prática a diferença fundamental entre comunicação direta e indireta em sistemas distribuídos. A implementação do padrão Publish-Subscribe no contexto da clínica veterinária mostrou que eventos do domínio — vencimentos, recalls, reposições — são naturalmente adequados ao modelo assíncrono baseado em tópicos.

As duas propriedades exigidas pelo enunciado foram demonstradas: o **desacoplamento espacial** foi evidenciado pelo fato de o `PublicadorClinica` não conter nenhuma referência a assinantes; o **desacoplamento temporal** foi comprovado pela retenção e posterior entrega de eventos publicados enquanto o assinante "Tecnico" estava offline.

A principal limitação da implementação atual é que o Broker opera in-process, sem interface de rede. A evolução natural seria expô-lo via HTTP (já projetado com `BrokerServidor.java`) ou adotar um broker externo como Apache Kafka, mantendo o mesmo contrato de tópicos definido pelo enum `Evento.Topico`.